Gate Before You Act

Presentation Script

Team Simpletons | Language Models and Agents (CL3.410)

Matches the slide prompts (v2) slide for slide. About 18–20 minutes at a calm pace (roughly 130 words a minute).
Speaker split: Varun slides 1–5, Aayush 6–10, Srushti 11–16, Dhruv 17–20. Change it as you like.

Slide 1: Title

Varun | ~30 s

Good morning. We are team Simpletons: Varun, Aayush, Srushti and Dhruv. Our project is called "Gate Before You Act". The question is simple to state: when an AI security agent wants to take a disruptive action, does checking its evidence, not just checking the rules, lead to better decisions? Everything we show today is in our public GitHub repository.

Slide 2: Domain, User and Problem

Varun | ~1 min 15 s

Our domain is security operations, and our user is a junior SOC analyst. Picture an alert at 2 a.m. One Windows machine is reading the memory of the login process, which is a classic password-stealing step. An AI agent reads the logs and says: isolate this machine.

Agents are starting to get tools like that: isolate a host, kill a process, disable an account. A recent study found frontier agents wrongly contain hosts in 45 to 82 percent of episodes. The usual safeguard is a policy gate, which asks whether the action is allowed and whether it needs approval. But it never asks whether the evidence actually points at this machine. The logs are partly written by the attacker. And in India, CERT-In requires reporting within six hours and keeping logs for 180 days, so mistakes and clean-ups have real consequences.

Slide 3: What Existing Work Misses

Varun | ~45 s

Existing work measures parts of this. OpenSec measures whether agents gather evidence, but it has no enforced gate. AgentSpec and ShieldAgent enforce rules at runtime, but rules only. CaMeL separates trusted and untrusted data to stop injection, but it doesn't judge evidence. Nobody isolates the value of checking evidence beyond policy. So we built a controlled test: same policy, same context, same model, same tools. Only the evidence check changes.

Slide 4: Research Question and Hypotheses

Varun | ~1 min

Our research question: does checking that the cited evidence supports the action improve decisions beyond a policy-only gate, and which part of the check helps?

We pre-registered three hypotheses with fixed thresholds. H1: the full gate wrongly admits at least 15 points fewer bad actions than the policy-only gate. H1-C4: the LLM verifier inside the gate accounts for at least 10 of those points on its own. H2 is the cost check: the full gate's unnecessary deferral should be no more than 10 points worse than policy-only.

Slide 5: Falsifiable Claim (course activity)

Varun | ~1 min

In class we wrote this as a falsifiable claim. Our system, the full evidence gate, will beat the policy-only gate on evidence packages that should be rejected. That is measured by the wrongly-admitted rate on our 31 frozen test scenarios, 124 cases. We'd consider it falsified if the improvement is under 15 points, or if the confidence interval includes zero.

Our baseline is the policy-only gate, along with a plain agent with no gate at all. The capability is reasoning plus retrieval inside a tool gate. The result, 29 points, was not falsified. As you'll see, our cost hypothesis was. Now Aayush will take you through the method.

Slide 6: Four Capabilities

Aayush | ~1 min 15 s

The brief asks for several capabilities, each one needed and measured. We use four.

Tool calling: the agent acts only through tools, and every action is gated. We measure tool-call accuracy and the unsafe-action rate, and we ablate by serving the same tools directly or over MCP. RAG: the verifier looks up Sigma detection rules and ATT&CK descriptions. We test three levels, no RAG, simple RAG and RAG with a reranker, with standard retrieval metrics. Reasoning: an LLM verifier judges whether the evidence supports the action. We remove it, or let it see the agent's explanation. Agentic: after an "insufficient" verdict the agent decides whether to search again, act or hand off to a human. We vary the recovery budget between zero, one and two searches and measure task success.

We did not fine-tune. Our question is about gate design with the model held fixed.

Slide 7: Architecture

Aayush | ~1 min

Here is GateBench. Logs, shown in red, are untrusted. The agent investigates, then proposes one action and cites the log records that justify it. The proposal has to pass the gate. C1 checks the allow-list, and that every identifier came from a record the agent actually retrieved, which blocks injected targets. C2 checks citation scope. C3 checks in code that the target appears as the actor. C4 is the LLM verifier. C5 is policy, and C6 handles approvals. Trusted context, in green (asset tiers, accounts, approvals, change tickets), comes from a trusted source, never from the logs.

Slide 8: One Tool's Schema (course activity)

Aayush | ~1 min

In class we wrote the schema for one of our tools. Here is kill_process. Use it when a cited record shows this process itself doing the malicious activity. Don't use it for the parent, for the target of an access such as lsass, or for system processes; escalate instead.

Its parameters are host and pid. The values a model is likely to get wrong are an IP address instead of a host name, a PID passed as a string, the access mask 0x1438 from the same record, or the parent's PID. And these are the exact error strings the agent gets back, for example: "PID 7924 does not appear as a PID in any record you have retrieved."

Slide 9: Evaluation Set

Aayush | ~1 min 15 s

Our evaluation set is built from 99 real attack recordings, about 750,000 log events. For each scenario we write an analyst request, trusted context and a correct evidence package. A builder then edits the log database to create harder variants: harmless citations, the attack moved to the wrong host, a partial or contradicted chain, or activity covered by an approved maintenance ticket. That makes 31 test scenarios and 217 cases, each with an exact right answer.

To be transparent: the labels were produced by separate AI agent sessions following our written rules. The second annotator labelled blind, and a third session adjudicated. Agreement on the labels our hypotheses use is high, and the scoring itself uses no LLM judge.

Slide 10: Baselines and Setup

Aayush | ~45 s

We have two baselines: a plain agent with no gate, and the policy-only gate. Then come the full gate and eight ablations, each differing in exactly one setting. Everything ran on one 8 GB laptop GPU with a 4-bit Qwen 7B model: four runs with zero errors. We used under three GPU-hours of the 81 we planned, and there was no API cost. Srushti will now show the results.

Slide 11: Headline Results

Srushti | ~1 min

Here are the three hypotheses with their confidence intervals. H1 is supported: the full gate wrongly admits 29 points fewer bad actions, comfortably above our 15-point target. H1-C4 is supported: the LLM verifier on its own accounts for 16 points. But H2 is not met. Unnecessary deferral rises by 35 points, far past our 10-point margin. So the gate is safer, and also much more hesitant.

Slide 12: Experiment 1

Srushti | ~1 min

In Experiment 1 every gate sees the same fixed packages. With no gate, everything gets through. Policy alone still admits 57 percent of the bad packages. With evidence checks, the full gate admits 28 percent. Citation scope adds nothing over policy, and the code-based target check adds nothing once the verifier is there. The catch: the full gate wrongly blocks 35 percent of correct packages, while policy-only blocks none.

Slide 13: RAG Ablation

Srushti | ~1 min

Here is the RAG ablation at three levels. The reranker roughly doubles retrieval quality: Hit@5 goes from 6 to 13 percent. But the gate's decisions don't change. With no retrieval at all, wrongly admitted is the same 28 percent. Retrieval is weak because log text is full of file paths that confuse keyword search, and because the verifier's mistakes are about reasoning, not about missing background knowledge.

Slide 14: Agentic Ablation

Srushti | ~1 min 15 s

For agentic behaviour we varied the recovery budget. In the registered run the agent used recovery

only once in 88 chances, because its instructions also told it to escalate when evidence was thin. So we ran an exploratory run with just two prompt lines changed. Now it recovered in 20 of 29 cases, yet task success moved by at most about 2 points, and most of the actions it then took were unsafe. It found other records to cite. For communication, we served the tools over MCP with the gate inside the server, and every outcome matched within half a point. The gate doesn't depend on the transport.

Slide 15: Reasoning and Tool Calling

Srushti | ~1 min

The verifier is right 58 percent of the time, rising to 64 when it sees the agent's explanation. But it never once said "contradicted" in 2,604 calls, even when an approved ticket covered the activity. On the tool-calling side, tool-call accuracy is about 96 percent. Of the unsafe calls the agent proposed, policy-only let through 47 percent and the full gate 16 percent. A planted injection was proposed once and blocked, and nobody tried to delete logs.

Slide 16: Experiment 2

Srushti | ~1 min

In full episodes, where the agent acts over several steps, the gate cuts unsafe executions from 40 percent to 23. But unnecessary deferrals rise from 7 percent to 23. On packages that were actually correct, the full gate's agent defers 40 percent of the time. That's the over-blocking from Experiment 1 turning into "let me ask the analyst". Dhruv will take us through the failures and our audit.

Slide 17: Qualitative Examples and Failure Analysis

Dhruv | ~1 min 15 s

Here are four failures that explain the numbers. In one, the verifier writes that the maintenance ticket matches, and still says "supports": our schema made it give the verdict before comparing the ticket. In another, a correct action is called insufficient and the agent defers. In a third, the agent re-proposes the same call seven times instead of searching. In the fourth, a recovery search finds nothing, the agent cites other records, and an unsafe action is admitted. Across Experiment 2, over-deferral is tagged in 151 full-gate episodes against 46 for policy-only. In live investigation, every system found only 2 of 24 techniques.

Slide 18: Intake Questions (course activity)

Dhruv | ~1 min

We also ran the course's intake questions on our own agent. The one part that genuinely needs the model to choose the next step is the decision after gate feedback: search, act or hand off. Everything else is deterministic code. An episode costs about five model calls, or about 26 with live investigation. The tool we assume but don't have is a real EDR, Active Directory or firewall integration; ours are mocked. The worst action would be disabling a domain admin, which everyone would notice. Our policy requires approval for that, and the trace flags it.

Slide 19: Trifecta Audit (course activity)

Dhruv | ~1 min

And the lethal-trifecta audit. Private data: the logs and the trusted context. Untrusted content:

attacker-written command lines and file names, plus the reference rules. Egress: the action tools, analyst messages and reports, and traces. We cut egress to the outside world: actions are mocked, the model runs locally, and there's no web access and no rendered links. The cost to the demo is no real containment, no threat-intelligence lookups, and a small 7B model, which is exactly where our weak verifier comes from.

Slide 20: Limitations, Future Work, Conclusion

Dhruv | ~1 min 15 s

Our limitations, plainly. The labels were produced by AI agents and haven't yet been checked by people on a sample. We have 31 scenarios, one 7B model and mock tools, and blocking an IP couldn't be tested. H2 depends on adjudicated escalations, but a blind re-check agreed 95 percent and didn't change it.

To conclude: checking evidence works like a good brake. It cuts wrongly admitted actions by 29 points, and the LLM verifier contributes 16 of them. With today's small verifier, though, it also brakes when it shouldn't. Retrieval, recovery and MCP don't fix that, which points to the verifier itself. Next we would have it compare the ticket before its verdict, give the agent filled-in feedback, try stronger or fine-tuned verifiers, and validate labels with people. Thank you. We're happy to take questions.

Likely questions and short answers

Q: Why trust labels made by AI agents?
A: They were derived from written rules with a logged reason for each, the second annotator was blind, and no agent saw system behaviour while labelling. Agreement on the labels our hypotheses use is kappa 0.92 to 1.00. We still list human validation as the first piece of future work.

Q: Isn't H2 failing a failure of the project?
A: No. It's a measured trade-off, and the analysis shows why: the verifier is miscalibrated. Retrieval, recovery and MCP don't fix it, which narrows the problem to the verifier.

Q: Why no fine-tuning?
A: It would change the model, and our question holds the model fixed. The results now motivate fine-tuning the verifier as future work.

Q: Why wasn't A2A tested?
A: We tested communication as direct calls vs MCP, the protocol our tools are built on. A multi-agent A2A hand-off between investigator and responder is future work.

Q: Why did compute come in so far under budget?
A: The budget assumed every output hit its length cap; real outputs were much shorter, and vLLM batched requests efficiently.